

CS 3960 (3)

Vibe Coding

Spring 2026, *content* Pavel Panchekha and John Regehr, *illustrations* Gemini 3 Nano Banana Pro

- Make sure to do the reading assignment by Friday 5pm
 - Two things to read
- How's the latency assignment coming along?
- Latency demo day is Weds after spring break
 - Show us opening large.txt and doing a find and replace
 - Describe optimizations and final editor architecture
 - Be prepared to show and explain code



Today:

Intro to formal verification of
software — the idea that we can
mathematically prove things
about computer programs

AI is making formal verification
into a key concern rather than a
niche topic for researchers and
safety-critical systems





For example, we might want to prove that:

- Compressing and decompressing a file gives back the original bytes
- Operating system user A can never read memory or files belonging to user B
- Our application never crashes
- Our application never executes code besides what was shipped

Blueprint for all formal verification efforts:

1. Write a specification for what the program should or shouldn't do
2. Prove that the program meets the specification





Some specifications are pretty simple, E.g. “Don’t crash”

Other specifications are very, very complicated:

“Be a correct C++20 compiler”

“Be a correct air traffic control system”

We're not going to look at
specifications in very much detail
But... they're just math





**What does it mean to prove
something mathematically?**

We can understand programs
using math

However, the behavior of
programs often diverges from the
elementary math we learned in
school





In math, it is often the case that:

$$(x+y)+z = x+(y+z)$$

In IEEE floating point math:

$$(x+y)+z \neq x+(y+z)$$

In IEEE floating point math, we have both +0 and -0

What does $x+1$ mean in math,
where x is a natural number?

What are some of the things that
 $x+1$ might mean, where x is a
computer number?





In C and C++:

$$(x+1) > x$$

... is always true when x is a signed integer (but in a tricky way).

... but is sometimes false when x is an unsigned integer.

The point:

To prove things about computer programs, we have to have an extremely detailed understanding of what programming languages actually mean.

Seems like kind of bad news.





But there is good news:

We can offload understanding programming languages to tools.

Now the workflow is:

1. Write a specification in a dedicated specification language
2. Ask a tool to verify that our implementation meets this spec
3. (Probably) struggle with the tool for a while

We'll talk a bit more about tools later.

For now, let's reason about programs using our brains.

Doing lots of little proofs in your head is one of the best ways to write better software.





What do these little proofs look like?

At some point in your program, you'll want to know something like:

- This pointer is not null
- This array index is in bounds
- This file is open for reading
- This thread has exclusive access to a data structure

How do you prove these things?

You work backwards: Look at the code leading up to the point in the program you care about, and you reason about its effects.

You have to look at all possible ways to get there.





One of the core ways we reason about code formally is using a “Hoare triple”:

$$\{ P \} S \{ Q \}$$

$\{ P \}$ — a precondition

S — a statement

$\{ Q \}$ — a postcondition

Then we chain these together across statements

Let's prove an implementation of
“popcount” correct. This function
returns the number of set bits in its
argument.

`popcount(0) = ?`

`popcount(7) = ?`

`popcount(8) = ?`

`popcount(-1) = ?`





```
int popcount(uint64_t n) {  
    int count = 0;  
    while (n) {  
        n = n & (n - 1);  
        ++count;  
    }  
    return count;  
}
```


What's the specification?

Let's just assume we have a reference,
`popcount-ref()`.

So, we're trying to prove that

`popcount-ref(x) = popcount (x)`

... for all 2^{64} possible values of x



Precondition?

```
int popcount(uint64_t n) {
```

```
    int count = 0;
```

Postcondition?

```
    while (n) {
```

```
        n = n & (n - 1);
```

```
        ++count;
```

```
    }
```

```
    return count;
```

```
}
```

Precondition?

Postcondition?

```
int popcount(uint64_t n) {  
    int count = 0;  
    while (n) {  
        n = n & (n - 1);  
        ++count;  
    }  
    return count;  
}
```


Precondition?

Postcondition?

```
int popcount(uint64_t n) {  
    int count = 0;  
    while (n) {  
        n = n & (n - 1);  
        ++count;  
    }  
    return count;  
}
```

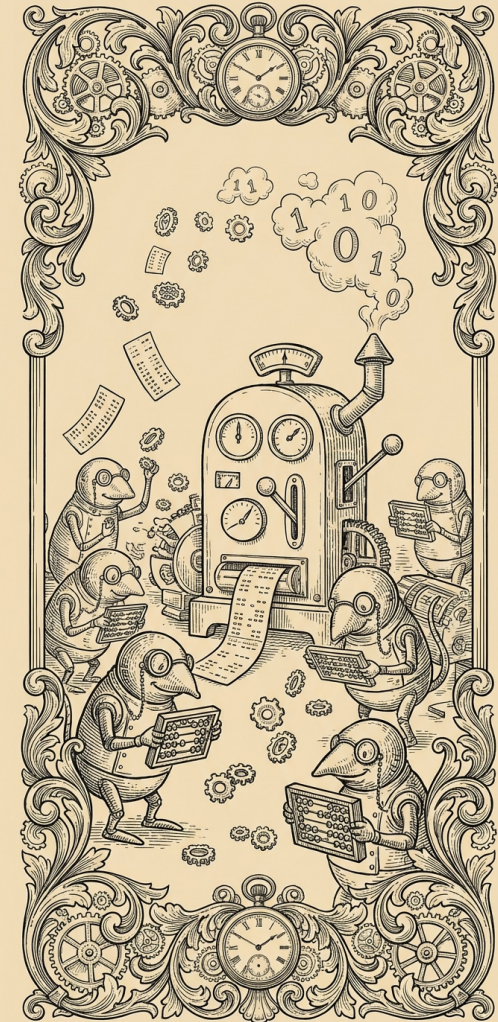
Why does $n \& (n - 1)$ clear
the lowest set bit?

????1000 $\leftarrow n$

????0111 $\leftarrow n - 1$

????0000 $\leftarrow n \& (n - 1)$

Higher bits are unaffected



Precondition?

Postcondition?

```
int popcount(uint64_t n) {  
    int count = 0;  
    while (n) {  
        n = n & (n - 1);  
        ++count;  
    }  
    return count;  
}
```


Ok we're kind of stuck...

From what we've seen to far,
Hoare logic won't get us any
further

The loop is the problem — we
need an insight about it, in order
to make progress





The usual way to reason about the purpose of a loop is to find a loop invariant:

A statement about the program that is true both before and after every loop iteration

count >= 0

count >= 0

```
int popcount(uint64_t n) {  
    int count = 0;  
    while (n) {  
        n = n & (n - 1);  
        ++count;  
    }  
    return count;  
}
```

```
int popcount(uint64_t n) {  
    int count = 0;  
    while (n) {  
        n = n & (n - 1);  
        ++count;  
    }  
    return count;  
}
```

What needs to be
true here?

What needs to be true
here?

count =
popcount-ref(orig n)

```
int popcount(uint64_t n) {  
    int count = 0;  
    while (n) {  
        n = n & (n - 1);  
        ++count;  
    }  
    return count;  
}
```

What can we say about
count before the last
loop iteration?

What needs to be true
here?

count =
popcount-ref(orig n)

```
int popcount(uint64_t n) {  
    int count = 0;  
    while (n) {  
        n = n & (n - 1);  
        ++count;  
    }  
    return count;  
}
```

count + 1 =
popcount-ref(orig n)

What needs to be true
here?

count =
popcount-ref(orig n)

```
int popcount(uint64_t n) {  
    int count = 0;  
    while (n) {  
        n = n & (n - 1);  
        ++count;  
    }  
    return count;  
}
```

```
int popcount(uint64_t n) {  
    int count = 0;  
    while (n) {  
        n = n & (n - 1);  
        ++count;  
    }  
    return count;  
}
```

count +
popcount-ref(n) =
popcount-ref(orig n)

What needs to be true
here?

count =
popcount-ref(orig n)

Why is this true
before the first loop
iteration?

count +
popcount-ref(n) =
popcount-ref(orig n)

Does this imply the
function postcondition?

```
int popcount(uint64_t n) {  
    int count = 0;  
    while (n) {  
        n = n & (n - 1);  
        ++count;  
    }  
    return count;  
}
```

How do we know that
count never overflows?

```
int popcount(uint64_t n) {  
    int count = 0;  
    while (n) {  
        n = n & (n - 1);  
        ++count;  
    }  
    return count;  
}
```



So far we've proved **partial correctness**: If the function terminates, it gets the correct answer

Total correctness requires adding a termination proof.

We often prove loop termination by finding a **loop variant** — think of it as fuel that is used by every loop iteration.

Our loop variant is “Every loop iteration decreases the number of set bits in n by 1.”





Is this kind of reasoning easy?

No.

Alas, it's the only way to formally verify software.

Big chunks of this work can be automated, but loop invariants remain difficult.

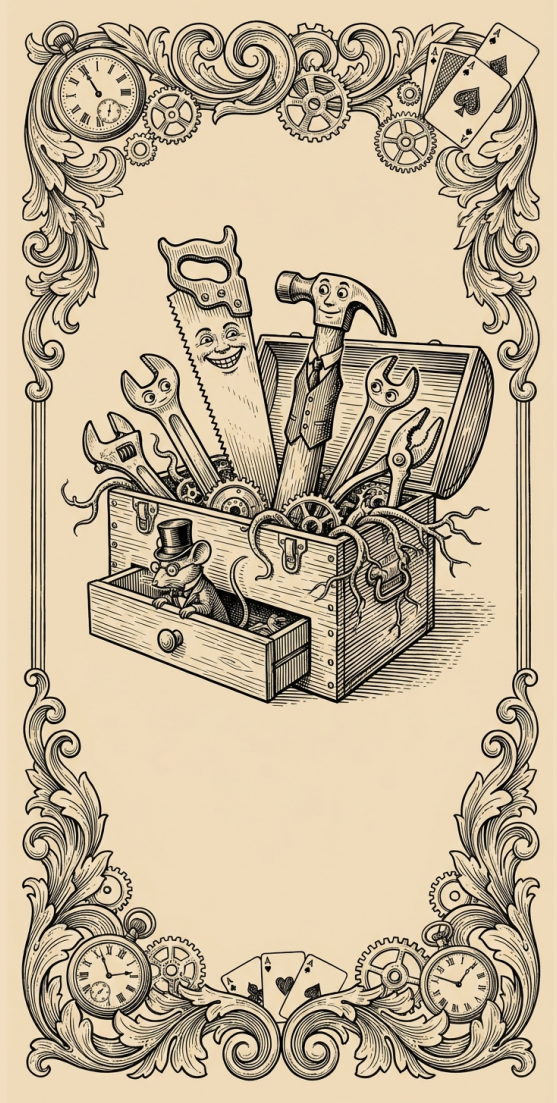
Example formal verification tools:

For Rust:

- Verus, Kani, Verifast

For C:

- Frama-C, CBMC, Verifast





Tools are nice, but the important thing is to be able to reason formally about code.

Further reading:

Loop invariants: analysis, classification, and examples